

Tufts HPC 集群使用教程

从零开始，以强化学习实验矩阵为例

目录

日常速查表	2
1 Quick Start: 科研工作者的日常流程	3
1.1 一次典型的工作流	3
1.2 每天重复的部分	4
1.3 遇到问题时怎么办	5
2 连接到集群	5
2.1 新旧集群: 确保你连的是新集群	5
2.2 SSH 登录	6
2.3 OnDemand 网页界面	6
2.4 进阶: 免密码登录与跳板机 (无需 VPN)	6
2.4.1 第一步: 生成密钥对 (如果还没有)	6
2.4.2 第二步: 把公钥传到集群上	6
2.4.3 第三步 (可选): 配置别名, 简化命令	7
2.4.4 第四步 (可选): 通过跳板机连接, 不需要 VPN	7
3 SLURM 详解: 提交和管理作业	8
3.1 作业脚本的结构	8
3.2 输出文件在哪里?	9
3.2.1 SLURM 日志文件 (标准输出/错误)	9
3.2.2 你的程序自己写出的文件	9
3.3 怎样取回结果文件?	10
3.4 提交和管理作业	11
3.5 交互式会话: 调试用	11

4 文件传输：怎样上传和下载文件？	12
4.1 场景一：传几个文件（用 scp）	12
4.2 场景二：传一个包含几十个文件的项目（用 rsync）	13
4.3 场景三：传输很大的数据集（用 Globus）	13
4.4 场景四：小文件快速上传（用 OnDemand）	14
4.5 文件传输方式总结	14
5 软件环境：怎样使用程序和包？	14
5.1 Module 系统：集群的“软件开关”	14
5.2 Python 环境管理：Conda（官方推荐方式）	15
5.2.1 第一步：配置存储路径（只需做一次）	15
5.2.2 第二步：创建和使用环境	16
5.2.3 在 SLURM 脚本中使用 Conda 环境	17
5.3 关于 uv：需要额外安装	17
5.3.1 正确的方式：先手动 sync，脚本里直接用.venv	18
6 存储系统：你的文件存在哪里？	19
6.1 登录后看到的文件都是谁的？	19
6.2 两类存储空间	20
6.3 集群禁止存放受限数据	21
7 目录管理：怎样组织你的文件？	21
7.1 推荐的目录布局	21
7.2 核心原则	21
7.3 实用命令	22
8 集群资源概览	23
8.1 分区（Partition）	23
8.2 每用户资源限制	23
8.3 可用 GPU	23
8.4 怎样查看集群当前的资源占用情况？	24
8.4.1 方法一：hpctools（推荐，最直观）	24
8.4.2 方法二：sinfo（命令行快速查看）	24
8.4.3 方法三：OnDemand 网页界面	25
8.4.4 查看正在运行的作业的实时资源使用	25
9 SLURM 的核心哲学：理解 HPC 的运行方式	25
9.1 你的电脑 vs HPC 集群	25
9.2 SLURM 是什么？	26

目录	3
9.3 两种关键的节点：登录节点 vs 计算节点	26
9.4 你的程序到底是怎么“跑起来”的?	27
10 实战：强化学习实验矩阵	27
10.1 项目结构	28
10.2 配置文件	28
10.3 训练脚本接口	28
10.4 方案一：SLURM Array Job（推荐）	29
10.4.1 提交脚本	31
10.4.2 控制并发数	31
10.5 方案二：嵌套循环提交	31
11 实用技巧	33
11.1 利用 preempt 分区跑更多实验	33
11.2 用邮件通知监控实验	33
11.3 汇总实验结果	34
11.4 检查哪些实验失败了	34
12 完整 workflow 总结	35
13 附录 A：常用 SLURM 命令速查	36
14 附录 B：常用 #SBATCH 指令速查	36
15 附录 C：致谢声明	36
16 附录 D：使用规则——初学者容易踩的坑	36

日常速查表

以下是你在集群上最常用的操作，不需要翻教程正文，直接复制粘贴即可。

连接与节点

操作	命令
SSH 登录	<code>ssh your_utln@login-prod.pax.tufts.edu</code>
快速开一个 CPU 计算节点	<code>srun -p batch -t 0-2:00:00 -n 2 --mem=4g --pty bash</code>
快速开一个 GPU 计算节点	<code>srun -p gpu -t 0-2:00:00 -n 2 --mem=4g --gres=gpu:1 --pty bash</code>
退出计算节点	<code>exit</code>

提示

很多操作（查目录大小、安装 Conda 环境等）在登录节点上会很慢，**开一个 CPU 计算节点再操作会快很多。**

作业管理

操作	命令
提交作业	<code>sbatch script.sh</code>
查看我的作业	<code>squeue --me</code>
取消某个作业	<code>scancel JOBID</code>
取消我的所有作业	<code>scancel -u \$USER</code>
查看已完成作业的效率	<code>seff JOBID</code>

资源与存储

文件传输（在本地电脑上运行）

环境管理

操作	命令
查看我的配额（实时刷新）	<code>watch -n 5 quota -s</code>
查看 Home 目录各文件夹大小	<code>du -sh ~/* ~/.* 2>/dev/null sort -rh head -20</code>
查看集群 GPU 空闲情况	<code>module load hpctools && hpctools</code> （选 2）
查看各分区状态	<code>sinfo</code>
查看 GPU 使用（在计算节点上）	<code>nvidia-smi</code>

操作	命令
上传项目目录	<code>rsync -azP ./project/ user@login-prod.pax.tufts.edu:~/project/</code>
下载结果	<code>rsync -azP user@login-prod.pax.tufts.edu:~/project/results/ ./results/</code>

1 Quick Start: 科研工作者的日常流程

如果你只想尽快跑起来，这一节就够了。后面的章节是详细解释。

1.1 一次典型的工作流

假设你已经在本地写好了代码，想在集群上跑。整个过程只需要 5 步：

第 1 步：把代码同步到集群

在你的本地电脑上运行：

```
rsync -azP --exclude='.venv' --exclude='__pycache__' \
./my_project/ your_utln@login-prod.pax.tufts.edu:~/my_project/
```

如果你已经配置了 SSH 别名（见连接章节），可以简写为：

```
rsync -azP --exclude='.venv' --exclude='__pycache__' \
./my_project/ hpc:~/my_project/
```

`rsync` 只传有变化的文件，所以第二次开始会很快。

第 2 步：SSH 登录集群

```
ssh hpc # 或 ssh your_utln@login-prod.pax.tufts.edu
```

第 3 步：提交作业

假设你的项目里已经有一个 SLURM 脚本 `run.sh`：

```
cd ~/my_project
sbatch run.sh
```

操作	命令
加载 Conda	<code>module load miniforge/25.3.0</code>
激活环境	<code>conda activate rl_env</code>
清理 uv 缓存（容易撑满配额）	<code>rm -rf ~/.cache/uv</code>
清理 pip 缓存	<code>rm -rf ~/.cache/pip</code>
清理 conda 包缓存	<code>conda clean --all -y</code>

```
# 输出: Submitted batch job 347502
```

不知道怎么写 SLURM 脚本? 这里是一个最小的模板, 保存为 `run.sh`:

```
#!/bin/bash
#SBATCH -J my_job           # 作业名
#SBATCH --time=00-06:00:00 # 最多跑 6 小时
#SBATCH -p gpu             # 使用 GPU 分区
#SBATCH --gres=gpu:1       # 1 块 GPU
#SBATCH --cpus-per-task=4  # 4 个 CPU 核
#SBATCH --mem=16g          # 16GB 内存
#SBATCH --output=%x.%j.out # 标准输出日志
#SBATCH --error=%x.%j.err  # 错误日志

module load miniforge/25.3.0 # 加载 Conda
module load cuda/12.9.0     # 加载 CUDA
conda activate my_env       # 激活你的环境

python train.py             # 运行你的程序
```

如果不需要 GPU, 把 `-p gpu` 改成 `-p batch`, 删掉 `--gres` 那行。

第 4 步: 查看作业状态

```
squeue --me           # 看作业是在跑(R)还是排队(PD)
cat my_job.347502.out # 看输出日志 (347502 换成你的 Job ID)
```

提交之后你可以断开 SSH 去做别的事, 作业会在后台继续跑。

第 5 步: 下载结果

作业跑完后, 在你的本地电脑上运行:

```
rsync -azP hpc:~/my_project/results/ ./results/
```

1.2 每天重复的部分

上面 5 步里, 第 2-4 步是你每天都会做的。最常见的循环是:

- 1. 本地改代码
- 2. rsync 同步到集群 (10 秒)
- 3. ssh hpc, sbatch run.sh (5 秒)
- 4. 过一会儿 squeue --me 看状态
- 5. 跑完后 rsync 拉结果回来

1.3 遇到问题时怎么办

情况	怎么做
作业一直排队 (PD)	sinfo 看分区是否有空闲节点; 减少资源请求; 尝试加 preempt 分区
作业运行失败	看 .err 日志文件找报错信息
不确定环境对不对	开一个交互式节点调试: srun -p batch -t 0-1:00:00 --mem=4g --pty bash
配额满了	watch -n 5 quota -s 监控; du -sh ~/.cache/* 找大文件; rm -rf ~/.cache/uv 清缓存
想取消所有作业	scancel -u \$USER

2 连接到集群

2.1 新旧集群：确保你连的是新集群

Tufts 目前有两套 HPC 集群——旧集群正在逐步淘汰，新集群（升级版）是官方推荐使用的。两者的登录地址不同：

	SSH 登录地址	OnDemand 网址
新集群（推荐）	login-prod.pax.tufts.edu	ondemand-prod.pax.tufts.edu
旧集群（淘汰中）	login.pax.tufts.edu	ondemand.pax.tufts.edu

注意

请始终使用新集群（带 -prod 后缀的地址）。旧集群仍然可以登录，但 Tufts 官方强烈建议迁移到新集群，旧集群将来会被关闭。如果你之前的脚本或配置里用了 login.pax.tufts.edu（不带 -prod），请尽快更新。

本教程所有内容均基于新集群。

2.2 SSH 登录

通过 SSH 连接到新集群的登录节点：

```
ssh your_utln@login-prod.pax.tufts.edu
```

这里 `your_utln` 是你的 Tufts 用户名（全小写）。如果你在校外，需要先连接 Tufts VPN（或使用跳板机，见后文）。

登录成功后，你会看到类似这样的提示符：

```
[your_utln@login-p01 ~]$
```

`login-p01` 表示你当前在第 1 个登录节点上。这个编号不重要，3 个登录节点看到的文件完全一样。

2.3 OnDemand 网页界面

如果你不熟悉命令行，也可以通过浏览器访问：

<https://ondemand-prod.pax.tufts.edu/>

OnDemand 提供图形化的文件管理器、终端、以及 Jupyter Notebook 等应用，适合入门和调试。

2.4 进阶：免密码登录与跳板机（无需 VPN）

每次登录都要输密码很烦。你可以配置 SSH Key 实现免密登录。

2.4.1 第一步：生成密钥对（如果还没有）

在你的本地电脑上运行：

```
# 检查是否已有密钥
ls ~/.ssh/id_*.pub

# 如果没有，生成一对
ssh-keygen -t ed25519
# 一路回车即可（不设密码短语）
```

2.4.2 第二步：把公钥传到集群上

```
ssh-copy-id your_utln@login-prod.pax.tufts.edu
```

输入一次密码后，以后就不需要了。验证：


```
ssh your_utln@login-prod.pax.tufts.edu
# 应该直接进去，不再问密码
```

提示

如果免密登录不生效，检查集群上的文件权限：

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

2.4.3 第三步（可选）：配置别名，简化命令

每次输 `ssh your_utln@login-prod.pax.tufts.edu` 太长了。在本地编辑 `~/.ssh/config`，添加：

```
Host hpc
    HostName login-prod.pax.tufts.edu
    User your_utln
```

之后只需要 `ssh hpc` 即可登录，`rsync`、`scp` 也可以用这个别名：

```
rsync -azP ./project/ hpc:~/project/
scp results.csv hpc:~/
```

2.4.4 第四步（可选）：通过跳板机连接，不需要 VPN

如果你在校外又不想用 VPN，可以通过一台**校内能访问的服务器**作为跳板（ProxyJump）。前提是你有一台可以从外网访问的校内机器（比如通过 ngrok 暴露的实验室服务器）。

在本地 `~/.ssh/config` 中配置：

```
# 跳板机（你的校内服务器，通过 ngrok 等方式从外网可达）
Host my_jump_server
    HostName your_server_address
    User your_username
    Port your_port

# HPC，通过跳板机连接
Host hpc
    HostName login-prod.pax.tufts.edu
    User your_utln
    ProxyJump my_jump_server
```

ProxyJump 告诉 SSH：先连跳板机，再从跳板机连 HPC。对你来说就是一条命令：

```
ssh hpc      # 自动经过跳板机，直达 HPC 登录节点
```

配合免密登录（对跳板机和 HPC 都做 `ssh-copy-id`），整个过程零密码、无 VPN。

```
# 分别对跳板机和 HPC 配置免密（各输一次密码，之后永久免密）
```

```
ssh-copy-id my_jump_server
```

```
ssh-copy-id hpc
```

注意

跳板机方案的前提是你有一台校内服务器可以从外网访问。如果你没有这样的机器，还是需要使用 Tufts VPN。

3 SLURM 详解：提交和管理作业

现在你已经了解了集群的基本结构、文件存储、以及怎么准备软件环境。接下来我们来看怎么真正提交一个计算任务。

3.1 作业脚本的结构

一个 SLURM 作业脚本就是一个普通的 Bash 脚本，但开头多了一些 `#SBATCH` 开头的特殊注释。SLURM 读这些注释来了解你需要什么资源。

我们来逐行看一个完整的例子：

```
#!/bin/bash
```

这行声明这是一个 Bash 脚本，是所有 shell 脚本的标准开头。

```
#SBATCH -J my_experiment      # 作业名称（方便你在队列里识别）
```

```
#SBATCH --time=00-06:00:00    # 最多运行 6 小时（格式：DD-HH:MM:SS）
```

`-J` 设置作业名称，会显示在 `squeue` 输出里。`--time` 设置时间上限——超过这个时间作业会被强制终止。所有公共分区的时间上限是 2 天（2-00:00:00）。

```
#SBATCH -p gpu                # 使用 gpu 分区
```

```
#SBATCH --gres=gpu:1          # 需要 1 块 GPU
```

```
#SBATCH --cpus-per-task=4      # 需要 4 个 CPU 核
```

```
#SBATCH --mem=16g              # 需要 16GB 内存
```

这几行声明你需要的**硬件资源**。`-p` 选择分区（后面会详细讲），`--gres` 请求 GPU，`--cpus-per-task` 和 `--mem` 分别请求 CPU 和内存。

```
#SBATCH --output=my_exp.%j.out # 标准输出写入此文件
#SBATCH --error=my_exp.%j.err  # 错误输出写入此文件
```

这两行**非常重要**——它们决定了你的程序输出（`print` 的内容、报错信息）保存在哪里。`%j` 会被替换为作业 ID，这样每次运行都有独立的日志文件。

```
# ---- 以下是普通的 shell 命令 ----
module load cuda/12.9.0      # 加载 CUDA
module load miniforge/25.3.0 # 加载 Conda
conda activate rl_env        # 激活环境

python train.py --lr 0.001    # 运行你的程序
```

`#SBATCH` 行之后，就是普通的 `shell` 命令，和你在终端里敲的一模一样。

注意

`#SBATCH` 行看起来像注释，但 SLURM 会解析它们。它们**必须紧跟在 `#!/bin/bash` 之后**，在任何非注释命令之前。如果中间插了一行普通命令（哪怕是 `echo`），后面的 `#SBATCH` 都会被忽略。

3.2 输出文件在哪里？

这是很多新手最常问的问题。答案取决于你怎么写 `--output` 和 `--error`：

3.2.1 SLURM 日志文件（标准输出/错误）

- 如果你写了 `--output=my_exp.%j.out`，日志文件就在**你提交作业时所在的目录下**
- 如果你写了 `--output=logs/%x.%j.out`，就在那个目录的 `logs/` 子目录下（目录必须提前创建）
- 如果你**没写** `--output`，默认输出到 `slurm-JOBID.out`，也在提交目录下

常用的文件名变量：

3.2.2 你的程序自己写出的文件

除了 SLURM 日志，你的程序通常还会写出其他文件（模型权重、CSV 结果等）。这些文件**保存在你程序里指定的路径下**。

例如，如果你的 `train.py` 里写了：

```
torch.save(model, "results/model.pt")
```

变量	含义
%j	Job ID (如 296794)
%x	作业名称 (-J 设置的名字)
%A	Array Job 的主 ID
%a	Array Task 的序号
%N	运行节点名

那么 `results/model.pt` 就在计算节点执行脚本时的工作目录下——通常就是你提交 `sbatch` 时所在的目录。

提示

最佳实践：在程序里用绝对路径或相对于项目根目录的路径来保存结果，避免文件散落到意想不到的地方。例如：

```
output_dir = f"/cluster/tufts/mylab/myutln/project/results/{env}/{arch}/
seed_{seed}"
os.makedirs(output_dir, exist_ok=True)
```

3.3 怎样取回结果文件？

作业跑完后，结果文件已经在集群的共享存储上了。你有几种方式获取它们：

方式一：直接在集群上查看

```
# SSH 登录后直接查看
cat my_exp.296794.out          # 看日志
ls results/                    # 看结果文件
```

方式二：下载到本地（小文件用 `scp`）

```
# 在你的本地电脑上运行
scp your_utln@login-prod.pax.tufts.edu:~/project/results/summary.csv ./
```

方式三：下载到本地（大量文件用 `rsync`）

```
# 只同步有变化的结果文件，增量下载
rsync -azP your_utln@login-prod.pax.tufts.edu:~/project/results/ ./results/
```

方式四：超大数据用 Globus

如果结果文件有几十 GB（比如大量 checkpoint），用 Globus 后台传输最稳定。参见第 4 节。

3.4 提交和管理作业

提交作业：

```
$ sbatch my_job.sh
Submitted batch job 296794      # SLURM 返回 Job ID, 记住这个号
```

查看作业状态：

```
$ squeue --me
JOBID PARTITION NAME      USER ST TIME  NODES NODELIST
296794      gpu  my_exp yzhang R  3:42      1 pax003
```

状态码含义：R = 正在运行，PD = 在排队等资源，CG = 即将完成。

常见排队原因：Resources（资源不足）、Priority（优先级低）、QOSMax（超出配额限制）。

取消作业：

```
scancel 296794      # 取消指定作业
scancel -u $USER    # 取消你的所有作业
```

查看已完成作业的资源效率：

```
sacct 296794
```

这会显示你实际用了多少 CPU 和内存。如果利用率很低，下次可以减少请求量——请求越精确，排队时间越短。

3.5 交互式会话：调试用

如果你想在计算节点上直接敲命令（比如调试程序、测试环境），可以用 `srun` 申请一个交互式会话：

```
# CPU 交互式会话
srun -p batch -t 0-2:00:00 -n 2 --mem=4g --pty bash

# GPU 交互式会话
srun -p gpu -t 0-2:00:00 -n 2 --mem=4g --gres=gpu:1 --pty bash
```

`--pty bash` 的意思是“给我一个交互式的 bash 终端”。进入后你会看到提示符从 `login-p01` 变成类似 `pax006` 的计算节点名，说明你已经在计算节点上了。用 `exit` 退出并释放资源。

提示

退出时你可能会看到类似这样的报错：

```
srun: error: pax017: task 0: Exited with exit code 1
```

这通常是无害的，不代表出了什么问题。它只是说你的 shell 退出时状态码不是 0——最常见的原因是 Conda 的 (base) 环境激活脚本或之前某个命令（比如打错了一个命令名）在 shell 内部留下了非零退出码。资源已经正常释放了，不用担心。

提示

交互式会话的 QOS 允许最多 4 小时、1 个 GPU。适合调试，不适合跑完整实验。

4 文件传输：怎样上传和下载文件？

这是使用 HPC 的第一步，也是很多教程忽略的重要环节。

4.1 场景一：传几个文件（用 scp）

scp (Secure Copy) 是最简单的文件传输命令。它的语法和 cp 很像，只不过可以跨机器复制。

从本地上传到集群：

```
# 上传单个文件
scp train.py your_utln@login-prod.pax.tufts.edu:~/project/

# 上传整个目录（加 -r 参数）
scp -r my_project/ your_utln@login-prod.pax.tufts.edu:~/project/
```

冒号后面的部分是集群上的目标路径。~/ 表示你的 Home 目录。

从集群下载到本地：

```
# 下载单个文件
scp your_utln@login-prod.pax.tufts.edu:~/project/results.csv ./

# 下载整个结果目录
scp -r your_utln@login-prod.pax.tufts.edu:~/project/results/ ./local_results/
```

提示

scp 的格式是 scp 源路径目标路径，谁在前面谁就是“从哪里复制”。本地路径直接写，远程路径格式为 用户名 @ 主机：路径。

4.2 场景二：传一个包含几十个文件的项目（用 rsync）

当你的项目有很多文件时，`rsync` 比 `scp` 更好用，因为它有两个重要优势：

- **增量同步**：只传输有变化的文件，不会重复传输没改过的文件
- **断点续传**：如果中途断了，重新运行会从断点继续，不用从头来上传整个项目：

```
# -a: 保留文件属性  -z: 压缩传输  -P: 显示进度
rsync -azP ./my_experiment/ \
your_utln@login-prod.pax.tufts.edu:~/project/my_experiment/
```

下载实验结果：

```
rsync -azP your_utln@login-prod.pax.tufts.edu:~/project/results/ ./results/
```

排除不需要的文件（比如本地的虚拟环境或缓存）：

```
rsync -azP --exclude='.venv' --exclude='__pycache__' --exclude='.git' \
./my_experiment/ \
your_utln@login-prod.pax.tufts.edu:~/project/my_experiment/
```

提示

推荐工作流：每次修改完代码后，用 `rsync` 同步到集群。因为 `rsync` 只传改动的文件，所以即使项目很大也很快。

4.3 场景三：传输很大的数据集（用 Globus）

如果你需要传输几十 GB 甚至 TB 级别的数据，`scp` 和 `rsync` 都不太合适——它们走 SSH 通道，速度有限，而且长时间传输容易断。

Tufts 推荐使用 **Globus**，一个专门用于科研数据传输的平台：

1. 访问 <https://www.globus.org/>，用 Tufts 账号登录
2. 设置两个“端点”（Endpoint）：一个是你的电脑（安装 Globus Connect Personal），一个是 Tufts HPC 的 Collection
3. 在网页界面上选择源和目标文件，点击传输
4. Globus 在后台完成传输，支持断点续传，**不需要保持网络连接**

提示

Globus 的关键优势：“Fire and Forget”——发起传输后你可以关掉浏览器，它会在后台自动完成，完成后邮件通知你。使用 Globus 不需要 VPN。

4.4 场景四：小文件快速上传（用 OnDemand）

如果只是传个几 MB 的小文件，用 OnDemand 网页界面最方便：

1. 打开 <https://ondemand-prod.pax.tufts.edu/>
2. 点击 Files → Home Directory
3. 使用 Upload / Download 按钮

注意

OnDemand 文件传输限制为**每个文件不超过 976MB**。大文件请用 rsync 或 Globus。

4.5 文件传输方式总结

场景	推荐工具	优势	限制
几个小文件	scp	简单直接	不支持续传
项目目录同步	rsync	增量、续传	需要 SSH 连接
超大数据集	Globus	后台传输、不限大小	需要配置端点
临时小文件	OnDemand 网页	图形化、零配置	单文件 < 976MB

5 软件环境：怎样使用程序和包？

在你自己的电脑上，你可能直接 `pip install` 就行了。但在 HPC 上，软件管理方式不太一样。

注意

HPC 集群上你**没有管理员权限**，不能用 `apt-get`、`yum` 等系统包管理器安装软件。所有软件要么通过 Module 系统加载，要么通过 Conda/pip 安装到你自己的目录里。
另外，**不要在登录节点上跑任何计算或安装操作**。即使是创建 Conda 环境、安装 Python 包，也应该先用 `srun` 进入一个交互式计算节点再操作。

5.1 Module 系统：集群的“软件开关”

HPC 集群上预装了大量软件（Python、CUDA、GCC、R 等等），但它们默认**不在你的环境变量里**。你需要用 `module load` 命令来“激活”它们。

为什么要这样设计？因为不同用户可能需要不同版本的同一个软件。Module 系统让每个人可以精确控制自己使用哪个版本，互不干扰。

常用 module 命令：

```
# 查看有哪些软件可用（当前环境）
module av

# 搜索特定软件（搜索全部，包括有依赖关系的）
module spider python
module spider cuda

# 加载软件
module load miniforge/25.3.0 # 加载 Conda（推荐）
module load cuda/12.9.0     # 加载指定版本的 CUDA

# 查看当前加载了什么
module list

# 卸载某个软件
module unload miniforge

# 清空所有已加载的软件
module purge
```

提示

`module av` 只显示当前可以直接加载的软件。`module spider` 搜索范围更广，会告诉你某个软件是否存在、需要先加载哪些依赖。找软件时优先用 `module spider`。

5.2 Python 环境管理：Conda（官方推荐方式）

Tufts HPC 官方推荐使用 **Conda** 来管理 Python 环境和安装包。集群提供了两个 Conda module：

- `miniforge/25.3.0`（推荐）—更轻量
- `anaconda/2025.06.0` —更完整，预装更多包

5.2.1 第一步：配置存储路径（只需做一次）

Conda 环境和包缓存会占用大量空间。如果用默认路径，会很快撑满 Home 目录的 30GB 配额。必须先把 Conda 的存储路径指向 Research 存储：

```
# 先创建目录
mkdir -p /cluster/tufts/your_lab/$USER/condaenv
mkdir -p /cluster/tufts/your_lab/$USER/condapkg
```

```
# 配置 conda 使用这些目录
```

```
conda config --append envs_dirs /cluster/tufts/your_lab/$USER/condaenv/  
conda config --append pkgs_dirs /cluster/tufts/your_lab/$USER/condapkg/
```

这会在你的 Home 目录下创建一个 `~/.condarc` 配置文件。之后创建的所有环境和下载的包都会存储在 Research 存储里，而不是占用 Home 目录的配额。

注意

如果你在 **Home 目录** 下（而不是 Research 存储里）看到 `condaenv` 和 `condapkg` 文件夹，说明 Conda 的存储路径配置到了 Home 目录——这会很快撑满 30GB 配额。应该把它们迁移到 Research 存储：

```
# 把内容搬到 Research 存储
```

```
mv ~/condaenv/* /cluster/tufts/your_lab/$USER/condaenv/  
mv ~/condapkg/* /cluster/tufts/your_lab/$USER/condapkg/  
rmdir ~/condaenv ~/condapkg
```

```
# 然后更新 .condarc (参考上面的 conda config 命令)
```

5.2.2 第二步：创建和使用环境

```
# 进入交互式计算节点（不要在登录节点上操作！）
```

```
srun -p batch -t 0-2:00:00 -n 2 --mem=4g --pty bash
```

```
# 加载 Conda
```

```
module load miniforge/25.3.0
```

```
# 创建一个新环境
```

```
conda create -n rl_env python=3.11 -y
```

```
# 激活环境
```

```
conda activate rl_env
```

```
# 安装包（优先用 conda install，找不到的包再用 pip）
```

```
conda install pytorch gymnasium numpy pandas -c conda-forge
```

```
# 或者用 pip (conda 里找不到的包)
```

```
pip install some-rare-package
```

注意

不要混用 `conda install` 和 `pip install`——这可能破坏环境。最佳实践：先用 `conda install` 装所有能装的包，最后再用 `pip install` 补充 `conda` 里找不到的包。

5.2.3 在 SLURM 脚本中使用 Conda 环境

```
#!/bin/bash
#SBATCH -J my_job
#SBATCH --time=00-06:00:00
#SBATCH -p gpu
#SBATCH --gres=gpu:1

module load miniforge/25.3.0
module load cuda/12.9.0
conda activate rl_env

python train.py --lr 0.001
```

5.3 关于 uv：需要额外安装

如果你在本地习惯用 `uv` 管理 Python 项目，在 HPC 上也可以用，但需要注意：`uv` 不是集群预装的软件，没有对应的 `module`，你需要自己安装。

```
# 进入交互式计算节点
srun -p batch -t 0-2:00:00 -n 2 --mem=4g --pty bash

# 方法：先加载 Conda，用 pip 安装 uv 到用户目录
module load miniforge/25.3.0
pip install --user uv
```

安装后 `uv` 会被放在 `~/.local/bin/` 下。你需要确保这个路径在 `PATH` 里：

```
# 检查 uv 是否可用
which uv

# 如果找不到，手动添加 PATH（也可以写入 ~/.bashrc）
export PATH="$HOME/.local/bin:$PATH"
```

注意

以下事项需要注意：

- `uv` 不是 Tufts 官方支持的工具，遇到问题无法找 HPC 团队帮忙
- 计算节点可能没有外网访问权限，`uv sync` 需要下载包时可能会失败
- 如果外网受限，建议在登录节点完成 `uv sync`（仅限安装包，不要跑计算），或改用 Conda

5.3.1 正确的使用方式：先手动 `sync`，脚本里直接用 `.venv`

关键原则：不要在 SLURM 脚本里写 `uv sync`。

原因：如果你提交了一个 Array Job（比如 50 个任务），它们可能同时启动、同时执行 `uv sync`。每个任务都会尝试下载完整的依赖（比如 PyTorch 就好几个 GB），50 个任务同时下载 = 瞬间吃掉几十 GB 的缓存空间，直接撑爆你的存储配额。

正确做法是分两步走：

第一步：手动登录计算节点，运行一次 `uv sync`

```
# 开一个交互式节点
srun -p batch -t 0-1:00:00 --mem=4g --pty bash

# 进入项目目录，同步依赖（只需做一次，或依赖变化时重做）
cd ~/project/my_experiment
uv sync
```

这会在项目目录下创建 `.venv/`，里面包含所有依赖。因为所有节点共享同一个文件系统，这个 `.venv` 在任何计算节点上都能直接使用。

`uv sync` 会在 `~/.cache/uv/` 里缓存下载的包，下次 `sync` 时如果依赖没变就不需要重新下载。但这个缓存可能比较大（PyTorch 等大包会占几十 GB），如果 Home 配额吃紧，可以用 `rm -rf ~/.cache/uv` 清理。

第二步：SLURM 脚本里直接调用 `.venv` 中的 Python

```
#!/bin/bash
#SBATCH -J my_job
#SBATCH --time=00-06:00:00
#SBATCH -p gpu
#SBATCH --gres=gpu:1

module load cuda/12.9.0

cd ~/project/my_experiment
# 直接用 .venv 里的 python，不要 uv sync!
```

```
.venv/bin/python train.py
```

注意最后一行：`.venv/bin/python` 直接调用虚拟环境里的 Python 解释器，不需要 `uv run` 或 `uv sync`，也不需要 `source .venv/bin/activate`。

提示

推荐的 `uv` 工作流（完整版）：

1. 在本地开发，用 `uv add` 管理依赖
2. 用 `rsync` 把项目（包括 `pyproject.toml` 和 `uv.lock`）同步到集群
3. 在集群上手动 `srun` 进交互式节点，运行 `uv sync`（只需一次，或依赖变化时重做）
4. SLURM 脚本里用 `.venv/bin/python` 直接运行，**不要写 `uv sync`**

如果遇到网络问题导致 `uv sync` 失败，建议回退到 Conda 方案。

6 存储系统：你的文件存在哪里？

理解存储结构是使用 HPC 的基础。集群有一个 3PB 的共享存储系统，所有登录节点和计算节点都能访问。

6.1 登录后看到的文件都是谁的？

这是很多新手的困惑：登录集群后 `ls` 一下，看到一堆文件夹，不确定哪些是自己的、哪些是别人的、能不能删。

简单的答案是：你登录后看到的所有文件都是你自己的。

原因如下：

- 你 SSH 登录后，默认落在你的 **Home 目录** (`/cluster/home/your_utln`)
- 这个目录是**你的私人空间**，只有你能看到和修改（除非你主动改了权限）
- 其他用户的 Home 目录在 `/cluster/home/别人的 utln`，和你的完全隔离
- 即使你 `srun` 进入计算节点，看到的文件也一样——因为所有节点共享同一个文件系统，你还是在你自己的 Home 目录里

你可以用以下命令确认自己在哪里、文件归谁：

```
pwd          # 查看当前目录（应该显示 /cluster/home/your_utln）
ls -la       # 查看文件详情，第 3 列是文件所有者
whoami       # 确认当前用户身份
```

`ls -la` 的输出中，第 3 列显示文件所有者，第 4 列显示所属组：

```
drwxr-xr-x  3 your_utln your_utln 4096 Mar 23 10:00 condaenv
drwxr-xr-x  5 your_utln your_utln 4096 Mar 23 10:00 condapkg
```

~~~~~

这里显示的是文件所有者，全是你自己

**提示**

常见的“遗留文件”及其来源：

- `condaenv / condapkg` —Conda 的环境和包缓存目录。如果它们在 **Home** 目录下，说明 **Conda 路径配置不对**——应该迁移到 Research 存储里（见软件环境章节），否则会很快撑满 30GB 配额
- `ondemand` —OnDemand 网页界面自动创建的工作目录，**不要删**
- `.ipynb` 文件—你之前用 Jupyter Notebook 创建的，确认不需要可以删
- 软件名目录（如 Wolfram Mathematica）—你之前使用过该软件时留下的，确认不需要可以删

这些都是你自己之前操作时产生的，不是别人的文件，放心处理。

## 6.2 两类存储空间

| 类型                 | 路径                                                  | 配额                | 用途        |
|--------------------|-----------------------------------------------------|-------------------|-----------|
| <b>Home 目录</b>     | <code>/cluster/home/<br/>your_utln</code>           | 30 GB（固定）         | 个人配置、小型代码 |
| <b>Research 存储</b> | <code>/cluster/tufts/<br/>lab_name/your_utln</code> | 至少 50 GB（可<br>扩展） | 实验数据、大型项目 |

Research 存储和 Home 目录的归属逻辑类似：`/cluster/tufts/lab_name/your_utln` 这个子目录是你的个人空间。但 `/cluster/tufts/lab_name/` 这一层是整个实验室共享的，你可能看到同事的目录（取决于权限设置）。

**注意**

Home 目录只有 30GB，很容易被 conda 环境和数据集撑满。**建议把实验项目、数据集、conda 环境都放在 Research 存储里。**

查看用量：`df -H /cluster/tufts/your_lab_name`

**提示**

如果你想确保自己的 Home 目录不被其他用户查看，可以运行：

```
chmod 700 ~
```

这会把权限设为“只有自己可读写执行”。

## 6.3 集群禁止存放受限数据

根据 Tufts HPC 政策，集群上**不允许存放受限数据** (Restricted Data)，包括 HIPAA、FERPA 等受保护的信息。更多使用规则见附录 D。

# 7 目录管理：怎样组织你的文件？

集群不像你的个人电脑——它的存储有配额限制，而且你可能同时跑几十个实验。花几分钟规划好目录结构，会在后续省掉很多麻烦。

## 7.1 推荐的目录布局

```

/cluster/home/your_utln/           # Home 目录 (30GB)
  .condarc                         # Conda 配置 (指向 Research 存储)
  .bashrc                         # Shell 配置
  privatemodules/                 # 自定义 module (如果需要的话)

/cluster/tufts/your_lab/your_utln/ # Research 存储 (50GB+)
  condaenv/                       # Conda 环境 (按官方推荐配置到这里)
  condapkg/                       # Conda 包缓存
  projects/                       # 所有实验项目
    rl_matrix/                    # 项目 1
      train.py
      configs/
      scripts/
      results/
      logs/
    nlp_finetune/                 # 项目 2
    ...
  datasets/                       # 共享数据集 (避免每个项目复制一份)
    mujoco_data/
    imagenet_subset/

```

## 7.2 核心原则

### Home 目录只放配置文件

`~/.condarc`、`~/.bashrc` 等配置文件很小，放 Home 没问题。但代码、数据、Conda 环境都应该放到 Research 存储里——Home 只有 30GB，一个 PyTorch 环境就能吃掉一半。

### 每个项目一个独立目录

和你在自己电脑上一样，每个实验项目放在独立的目录里，内部包含代码、配置、脚本、结果、日志。这样项目之间不会互相干扰，也方便用 `rsync` 同步。

### 结果和日志与代码分开

在项目目录里用 `results/` 和 `logs/` 子目录分别存放程序输出和 SLURM 日志。这样清理日志时不会误删代码，下载结果时也不用把整个项目都拉下来。

### 数据集单独存放、项目间共享

如果多个项目用同一份数据集，不要复制多份。在 Research 存储里建一个 `datasets/` 目录，各项目通过路径引用它。

### 定期清理

实验跑完、论文发了之后，把不需要的 checkpoint、日志和中间文件删掉释放空间。特别是 SLURM 的 `.out` 和 `.err` 日志——一次 Array Job 就能产生上百个日志文件。

## 7.3 实用命令

```
# 查看 Home 目录用了多少空间
du -sh ~

# 查看 Research 存储的配额和用量
df -H /cluster/tufts/your_lab

# 找出占空间最多的目录（排序显示前 10 个）
du -h --max-depth=2 ~ | sort -rh | head -10

# 批量删除旧日志（比如 30 天前的 .out 和 .err 文件）
find logs/ -name "*.out" -mtime +30 -delete
find logs/ -name "*.err" -mtime +30 -delete
```

### 注意

集群的存储系统**不提供备份**，但会保留最近 90 天的每日快照 (Snapshots)，可以恢复最近误删或误改的文件。详情可参考 Tufts 官方的 File Recovery 教程。如果你有重要的实验结果，建议自己也备份一份到本地或云端。



## 8 集群资源概览

### 8.1 分区 (Partition)

SLURM 把计算节点分成几组，叫做“分区”。不同分区有不同的用途和限制：

| 分区      | 时间上限 | 说明                                                     |
|---------|------|--------------------------------------------------------|
| batch   | 2 天  | 默认分区，仅 CPU。不允许请求 GPU。                                  |
| gpu     | 2 天  | GPU 分区。必须用 <code>--gres</code> 请求 GPU，不允许只跑 CPU 任务。    |
| preempt | 2 天  | 资源最多（包含教授贡献的节点），但作业可能被节点所有者 <b>抢占</b> ——你的作业会被直接 kill。 |

### 8.2 每用户资源限制

| 分区组合        | CPU 核数上限 | 内存上限     | GPU 上限 |
|-------------|----------|----------|--------|
| batch + gpu | 250      | 5,000 GB | 10     |
| preempt     | 1,000    | 8,000 GB | 20     |

#### 提示

可以同时指定多个分区做 fallback: `#SBATCH -p gpu,preempt`。这样如果 gpu 分区没资源，会自动尝试 preempt。

### 8.3 可用 GPU

集群有多种型号的 GPU：

| GPU 型号     | 显存     | 适用场景       |
|------------|--------|------------|
| T4         | 16 GB  | 小模型推理、轻量训练 |
| L40 / L40s | 48 GB  | 中等规模训练     |
| A100-40G   | 40 GB  | 大规模训练      |
| A100-80G   | 80 GB  | 大模型训练      |
| H200       | 141 GB | 最大规模任务     |

请求特定型号的 GPU：

```
#SBATCH --gres=gpu:a100:1      # 请求 1 块 A100 (不区分 40G/80G)
#SBATCH --constraint="a100-80G" # 加上这行可以指定 80G 版本
```

### 注意

除非你的框架支持多 GPU 并行，否则**不要请求多块 GPU**——你只会浪费资源、延长排队时间。

## 8.4 怎样查看集群当前的资源占用情况？

在提交作业之前，你可能想知道：现在哪个分区有空闲资源？GPU 还剩多少？这可以帮助你决定选哪个分区、请求什么类型的 GPU，避免排长队。

### 8.4.1 方法一：hpctools（推荐，最直观）

Tufts 提供了一个叫 hpctools 的辅助工具，用交互式菜单帮你查看各种信息：

```
module load hpctools
hpctools
```

运行后会显示一个菜单，输入编号选择功能：

| 选项 | 功能                   |
|----|----------------------|
| 1  | 查看各分区可用的 CPU 核数和内存   |
| 2  | 查看各分区可用的 GPU 资源      |
| 3  | 查看某日期之后你的已完成作业历史     |
| 4  | 查看你当前运行的作业详情         |
| 5  | 查看 Research 存储的配额和用量 |
| 6  | 查看指定目录的详细空间占用        |
| 7  | 查看 Home 目录的空间占用明细    |

比如你想在提交 GPU 作业前看看还有多少 GPU 可用，选 2 就能看到各分区的 GPU 空闲情况。想检查自己的存储快满了没有，选 5 或 7。

### 8.4.2 方法二：sinfo（命令行快速查看）

如果你不想进交互式菜单，sinfo 可以一行命令看到集群状态：

```
# 查看所有分区的节点状态概览
sinfo
```

```
# 只看 gpu 分区，显示空闲/占用/总计
```

```
sinfo -p gpu -o "%P %a %D %C"
```

输出中 %C 格式为 已分配/空闲/其他/总计的 CPU 核数。

查看 GPU 分区中各节点的具体 GPU 空闲情况：

```
# 显示每个节点的 GPU 分配情况
```

```
sinfo -p gpu -O NodeList,Gres,GresUsed,CPUsState,FreeMem
```

### 8.4.3 方法三：OnDemand 网页界面

如果你偏好图形化界面，OnDemand 提供了两个有用的页面：

- **Clusters → System Status**：查看整个集群各分区的节点状态
- **Jobs → Active Jobs**：查看你自己的作业运行情况，点击单个作业可以查看详情，已完成的作业还能在 Resources and I/O 标签下看到 CPU 和内存的效率

### 8.4.4 查看正在运行的作业的实时资源使用

如果你的作业已经在跑了，想看它实际占用了多少 CPU 和 GPU：

```
# SSH 到作业所在的节点（先用 squeue --me 找到节点名）
```

```
ssh pax006
```

```
# 查看你的进程的 CPU 和内存使用
```

```
top -u $USER
```

```
# 查看 GPU 使用情况（在有 GPU 的节点上）
```

```
nvidia-smi
```

`nvidia-smi` 会显示每块 GPU 的算力利用率、显存占用、以及正在使用 GPU 的进程。如果你发现 GPU 利用率很低（比如低于 30%），可能说明你的程序瓶颈在数据加载而不是计算——可以尝试增加 `--cpus-per-task` 来加速数据预处理。

## 9 SLURM 的核心哲学：理解 HPC 的运行方式

在开始任何操作之前，你需要先理解 HPC 集群和你的笔记本电脑有什么本质区别。

### 9.1 你的电脑 vs HPC 集群

在你自己的电脑上，运行一个 Python 程序很简单：打开终端，输入 `python train.py`，程序就在你的电脑上跑了。你是这台电脑的唯一用户，所有资源（CPU、内存、GPU）都归你一个人用。

HPC 集群完全不一样。它是一台由上百台服务器组成的“超级计算机”，同时有几十甚至上百个用户在使用。如果大家都直接运行程序，就像几百个人同时在一个厨房里做饭——必然乱套。

所以集群需要一个**调度系统**来维持秩序：你不能直接跑程序，而是要先“申请资源”，说明你需要多少 CPU、多少内存、用多长时间，然后系统帮你安排。

## 9.2 SLURM 是什么？

**SLURM** (Simple Linux Utility for Resource Management) 就是 Tufts 集群使用的调度系统。你可以把它想象成一个**餐厅的排号系统**：

1. **你告诉服务员**（写一个作业脚本）：“我需要一个 4 人桌、靠窗的位置、用 2 小时”
2. **系统排号**（SLURM 把你的请求放进队列）
3. **有位置了就安排你入座**（当集群有足够空闲资源时，SLURM 把你分配到某台服务器上）
4. **你吃完走人**（程序运行完毕，结果写入文件，资源释放）

整个过程是**异步的**——你提交作业后可以关掉终端去做别的事，程序会在后台自动运行。

## 9.3 两种关键的节点：登录节点 vs 计算节点

### 核心概念

当你 SSH 连接到集群时，你降落在**登录节点** (Login Node) 上。登录节点就像一个大厅，只用来做轻量操作：编辑文件、提交作业、查看结果。

真正跑计算的是**计算节点** (Compute Node) ——你通过 SLURM 申请之后才能使用它们。

**永远不要在登录节点上直接跑训练程序**，这会影响所有其他用户。

那么具体什么**可以在登录节点上做**，什么**不可以**？一个简单的判断标准：

| 可以在登录节点做               | 不可以在登录节点做                |
|------------------------|--------------------------|
| 编辑文件 (vim, nano 等)     | 运行训练脚本 (python train.py) |
| module load (只是修改环境变量) | 处理大型数据集                  |
| sbatch 提交作业            | 运行消耗大量 CPU/GPU 的程序       |
| squeue、sinfo 查看状态      | 编译大型软件                   |
| hpctools 查看资源          |                          |
| rsync/scp 传文件          |                          |
| conda create 创建环境 *    |                          |

\* 注：创建 Conda 环境和安装包涉及一定计算，官方建议在交互式计算节点上操作，但如果只是小规模安装（几个包），在登录节点上做通常也不会有问题。

Tufts 集群有 3 个登录节点（login-p01, login-p02, login-p03），通过统一入口 login-prod.pax.tufts.edu 自动分配。

## 9.4 你的程序到底是怎么“跑起来”的？

这是很多新手最困惑的地方，让我们一步步拆解：

1. **你的代码和数据**存放在集群的共享存储系统上（一个 3PB 的大硬盘，所有节点都能访问）
2. 你写一个**作业脚本**（就是一个 .sh 文件），里面说明：需要什么资源、要运行什么命令
3. 你用 sbatch 命令提交这个脚本
4. SLURM 找到一台空闲的计算节点，在上面**以你的身份**执行这个脚本
5. 因为所有节点共享同一个存储系统，计算节点**能直接读取你的代码和数据**——不需要额外复制
6. 程序的标准输出（print 的内容）被重定向到一个日志文件里
7. 程序产生的输出文件（模型、结果等）直接写在共享存储上，跑完后你在登录节点就能看到

### 提示

关键理解：所有节点看到的是同一套文件系统。你在登录节点上传的文件，计算节点立刻就能读取；计算节点写出的结果，你在登录节点立刻就能下载。不存在“从计算节点复制结果到登录节点”的步骤。

## 10 实战：强化学习实验矩阵

现在我们把前面学到的知识串起来，实现一个完整的实验 workflow。

我们的实验是一个三维矩阵：

- **维度 1**：多个 RL 环境（CartPole, LunarLander, HalfCheetah, ...）
- **维度 2**：多个算法/架构（PPO, SAC, DQN, ...）
- **维度 3**：每组跑  $N$  个随机种子（确保统计显著性）

目标：一键提交所有实验，自动记录结果，方便后续分析。

## 10.1 项目结构

先来看整个项目长什么样：

```
project/
  train.py          # 训练脚本，接受命令行参数
  configs/
    envs.txt        # 环境列表，每行一个
    archs.txt       # 架构列表，每行一个
  scripts/
    run_matrix.sh   # SLURM Array Job 脚本
    submit_all.sh   # 提交脚本
  results/          # 程序输出的结果
  logs/             # SLURM 日志文件
```

`results/` 存放你程序写出的模型和指标，`logs/` 存放 SLURM 的标准输出和错误日志。两者分开管理比较清晰。

## 10.2 配置文件

用纯文本文件列出实验参数，方便修改，也方便脚本读取：

`configs/envs.txt`

```
CartPole-v1
LunarLander-v2
HalfCheetah-v4
Hopper-v4
Walker2d-v4
```

`configs/archs.txt`

```
PP0
SAC
DQN
TD3
A2C
```

## 10.3 训练脚本接口

`train.py` 需要接受命令行参数，这样 SLURM 脚本才能通过参数控制每次运行的内容：

train.py (参数部分)

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--env", type=str, required=True)
parser.add_argument("--arch", type=str, required=True)
parser.add_argument("--seed", type=int, required=True)
parser.add_argument("--output-dir", type=str, default="results")
args = parser.parse_args()

# 结果保存到: results/{env}/{arch}/seed_{seed}/
```

关键设计：把环境、算法、种子都做成参数，这样同一个脚本就能覆盖所有实验组合。

## 10.4 方案一：SLURM Array Job (推荐)

SLURM Array Job 允许你一次提交一批结构相同但参数不同的作业。核心思路很简单：

1. 把三维矩阵（环境 × 架构 × 种子）展开为一个一维的数字索引 0, 1, 2, ..., N-1
2. SLURM 为每个索引创建一个独立的 Task，通过环境变量 SLURM\_ARRAY\_TASK\_ID 告诉脚本“你是第几号”
3. 脚本根据这个编号反推出对应的（环境, 架构, 种子）组合

假设有  $E$  个环境、 $A$  个架构、每组  $N$  个种子，总任务数 =  $E \times A \times N$ 。

scripts/run\_matrix.sh

```
#!/bin/bash
#SBATCH -J rl_matrix
#SBATCH --time=00-12:00:00
#SBATCH -p gpu,preempt
#SBATCH -N 1
#SBATCH -n 1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16g
#SBATCH --gres=gpu:1
#SBATCH --output=logs/%x_%A_%a.out
#SBATCH --error=logs/%x_%A_%a.err
#SBATCH --mail-type=END,FAIL
```

注意 `--output` 里用了 `%A` (主 Job ID) 和 `%a` (Task 序号), 这样每个 Task 有独立的日志文件。

```
# ---- 配置 ----
NUM_SEEDS=5

# 读取环境和架构列表到数组
mapfile -t ENVS < configs/envs.txt
mapfile -t ARCHS < configs/archs.txt

NUM_ENVS=${#ENVS[@]}
NUM_ARCHS=${#ARCHS[@]}
```

`mapfile` 把文本文件逐行读入 Bash 数组。这样我们可以用索引来访问每个元素。

```
# ---- 从 SLURM_ARRAY_TASK_ID 解码实验参数 ----
# 把一维索引还原为三维坐标, 类似于把数组下标转换为行列号
TASK_ID=${SLURM_ARRAY_TASK_ID}

SEED=$((TASK_ID % NUM_SEEDS))           # 最内层: 种子
TASK_ID=$((TASK_ID / NUM_SEEDS))
ARCH_IDX=$((TASK_ID % NUM_ARCHS))       # 中间层: 架构
ENV_IDX=$((TASK_ID / NUM_ARCHS))        # 最外层: 环境

ENV=${ENVS[$ENV_IDX]}
ARCH=${ARCHS[$ARCH_IDX]}

echo "=== Task ${SLURM_ARRAY_TASK_ID}: env=${ENV}, arch=${ARCH}, seed=${SEED}
===="
```

这段代码的逻辑和“把一个一维数组下标转成多维下标”完全一样(取余、整除), 如果你学过数组的线性化存储就会觉得很熟悉。

```
# ---- 加载环境并运行 ----
module load cuda/12.9.0
module load miniforge/25.3.0
conda activate rl_env

python train.py \
  --env "$ENV" \
  --arch "$ARCH" \
  --seed "$SEED" \
  --output-dir "results/${ENV}/${ARCH}/seed_${SEED}"
```



### 10.4.1 提交脚本

提交脚本负责计算总任务数，然后一条命令提交所有任务：

scripts/submit\_all.sh

```
#!/bin/bash
# 计算总任务数
NUM_ENVS=$(wc -l < configs/envs.txt)
NUM_ARCHS=$(wc -l < configs/archs.txt)
NUM_SEEDS=5

TOTAL=$((NUM_ENVS * NUM_ARCHS * NUM_SEEDS))
MAX_INDEX=$((TOTAL - 1))

echo "Submitting ${TOTAL} tasks (${NUM_ENVS} envs x ${NUM_ARCHS} archs x ${NUM_SEEDS} seeds)"

# 创建日志目录（必须提前存在，否则 SLURM 写日志会失败）
mkdir -p logs results

# 提交 Array Job
sbatch --array=0-${MAX_INDEX} scripts/run_matrix.sh
```

以 5 环境  $\times$  5 架构  $\times$  5 种子为例，共 125 个任务，一条命令全部提交。

### 10.4.2 控制并发数

集群对每用户有 GPU 数量限制。用 % 符号限制同时运行的 Array Task 数：

```
# 最多同时跑 10 个任务（因为 gpu 分区每用户限 10 块 GPU）
sbatch --array=0-124%10 scripts/run_matrix.sh
```

如果你同时用了 preempt 分区，上限可以到 20。

## 10.5 方案二：嵌套循环提交

如果不同实验需要不同的资源配置（比如 MuJoCo 环境需要更多内存），可以用脚本循环提交独立作业，对每组实验单独设置参数：

scripts/submit\_loop.sh

```
#!/bin/bash
```

```

NUM_SEEDS=5

while IFS= read -r ENV; do
    while IFS= read -r ARCH; do
        for SEED in $(seq 0 $((NUM_SEEDS - 1))); do
            # 根据环境选择资源配置
            if [[ "$ENV" == *"Cheetah"* || "$ENV" == *"Hopper"* || "$ENV" == *"Walker
            "* ]]; then
                MEM="32g"
                TIME="01-00:00:00"
            else
                MEM="8g"
                TIME="00-04:00:00"
            fi

            sbatch \
                -J "rl_${ENV}_${ARCH}_s${SEED}" \
                --time=${TIME} \
                -p gpu,preempt \
                --gres=gpu:1 \
                --cpus-per-task=4 \
                --mem=${MEM} \
                --output="logs/${ENV}_${ARCH}_s${SEED}.out" \
                --error="logs/${ENV}_${ARCH}_s${SEED}.err" \
                --wrap="module load cuda/12.9.0 && module load miniforge/25.3.0 && \
                        conda activate rl_env && \
                        python train.py --env ${ENV} --arch ${ARCH} --seed ${SEED} \
                        --output-dir results/${ENV}/${ARCH}/seed_${SEED}"

            done
        done < configs/archs.txt
    done < configs/envs.txt

```

`sbatch --wrap` 可以直接传入命令字符串作为作业内容，不需要单独写作业脚本文件。适合逻辑简单的场景。

## 11 实用技巧

### 11.1 利用 preempt 分区跑更多实验

preempt 分区的资源上限更高 (20 GPU vs 10 GPU)，但作业可能被节点所有者的任务抢占——SLURM 会在约 30 秒内 kill 你的进程。

应对策略是让你的程序支持断点续训：

在 train.py 中添加 checkpoint 逻辑

```
import os, torch

ckpt_path = f"{args.output_dir}/checkpoint.pt"

# 启动时检查是否有已保存的进度
if os.path.exists(ckpt_path):
    checkpoint = torch.load(ckpt_path)
    model.load_state_dict(checkpoint["model"])
    start_epoch = checkpoint["epoch"]
    print(f"Resumed from epoch {start_epoch}")
else:
    start_epoch = 0

# 训练过程中定期保存
for epoch in range(start_epoch, total_epochs):
    train_one_epoch(model, ...)
    if epoch % 10 == 0: # 每 10 个 epoch 保存一次
        torch.save({"model": model.state_dict(), "epoch": epoch}, ckpt_path)
```

这样即使作业被抢占，重新提交后也能从上次的进度继续，而不是从头开始。

### 11.2 用邮件通知监控实验

```
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=your.email@tufts.edu
```

对于 Array Job（一次提交上百个任务），建议只在失败时通知，否则你的邮箱会被淹没：

```
#SBATCH --mail-type=FAIL
```

### 11.3 汇总实验结果

所有实验跑完后，用一个简单的 Python 脚本汇总各组的结果：

```
collect_results.py

import os, json, pandas as pd

rows = []
for env in open("configs/envs.txt"):
    env = env.strip()
    for arch in open("configs/archs.txt"):
        arch = arch.strip()
        for seed in range(5):
            result_file = f"results/{env}/{arch}/seed_{seed}/metrics.json"
            if os.path.exists(result_file):
                with open(result_file) as f:
                    metrics = json.load(f)
                    rows.append({"env": env, "arch": arch, "seed": seed,
                                **metrics})

df = pd.DataFrame(rows)
summary = df.groupby(["env", "arch"]).agg(
    reward_mean=("final_reward", "mean"),
    reward_std=("final_reward", "std"),
).reset_index()
summary.to_csv("results/summary.csv", index=False)
print(summary.to_string())
```

### 11.4 检查哪些实验失败了

```
# 查看有错误输出的日志
grep -l "Error\|Traceback\|CANCELLED" logs/*.err

# 查看哪些结果文件缺失
python -c "
import os
envs = open('configs/envs.txt').read().split()
archs = open('configs/archs.txt').read().split()
for e in envs:
    for a in archs:
```

```

    for s in range(5):
        p = f'results/{e}/{a}/seed_{s}/metrics.json'
        if not os.path.exists(p):
            print(f'MISSING: {e} / {a} / seed {s}')

```

## 12 完整 workflows 总结

1. 本地准备：开发代码，准备好 train.py、配置文件和 SLURM 脚本
2. 上传到集群：

```

rsync -azP --exclude='.venv' --exclude='__pycache__' \
    ./my_project/ your_utln@login-prod.pax.tufts.edu:~/project/

```

3. SSH 登录，设置环境：

```

ssh your_utln@login-prod.pax.tufts.edu
module load miniforge/25.3.0 && module load cuda/12.9.0
conda activate rl_env

```

4. 交互式调试——先跑一个任务确认一切正常：

```

srun -p gpu -t 0-1:00:00 --gres=gpu:1 --mem=8g --cpus-per-task=4 --pty
bash
python train.py --env CartPole-v1 --arch PPO --seed 0 --output-dir
    test_run
exit

```

5. 用 seff 检查资源使用，调整请求量
6. 批量提交所有实验：

```

bash scripts/submit_all.sh

```

7. 监控进度：

```

squeue --me                                # 查看状态
watch -n 60 'squeue --me | wc -l'          # 持续监控剩余任务数

```

8. 下载结果：

```
# 在本地运行
rsync -azP your_utln@login-prod.pax.tufts.edu:~/project/results/ ./
    results/
python collect_results.py
```

## 13 附录 A: 常用 SLURM 命令速查

| 命令                                   | 功能                  |
|--------------------------------------|---------------------|
| <code>sbatch script.sh</code>        | 提交批处理作业             |
| <code>squeue --me</code>             | 查看自己的作业             |
| <code>scancel JOBID</code>           | 取消指定作业              |
| <code>scancel -u \$USER</code>       | 取消自己的所有作业           |
| <code>seff JOBID</code>              | 查看已完成作业的 CPU/内存效率   |
| <code>sacct -j JOBID</code>          | 查看作业详细信息            |
| <code>scontrol show job JOBID</code> | 查看作业完整配置            |
| <code>sinfo</code>                   | 查看集群节点状态            |
| <code>nvidia-smi</code>              | 查看 GPU 使用情况（在计算节点上） |

## 14 附录 B: 常用 #SBATCH 指令速查

## 15 附录 C: 致谢声明

如果你的研究使用了 Tufts HPC 集群，请在论文中加入致谢：

*The authors acknowledge the Tufts University High Performance Compute Cluster (<https://it.tufts.edu/high-performance-computing>) which was utilized for the research reported in this paper.*

## 16 附录 D: 使用规则——初学者容易踩的坑

HPC 集群受 Tufts *Use of Information Systems Policy* 约束（完整版：[https://go.tufts.edu/hpc\\_aup](https://go.tufts.edu/hpc_aup)）。下面不列那些显而易见的规则，只说初学者容易犯的错误：

### 不要把账号密码分享给同事

即使是同一个实验室的人，也不能共用账号。每个人必须用自己的 UTLN 登录。如果你的同事需要访问集群，让他们自己申请账号。

| 指令                         | 说明                        |
|----------------------------|---------------------------|
| #SBATCH -J name            | 作业名称                      |
| #SBATCH --time=DD-HH:MM:SS | 最大运行时间（所有公共分区上限 2 天）      |
| #SBATCH -p partition       | 分区（可逗号分隔多个，如 gpu,preempt） |
| #SBATCH -N 1               | 节点数（通常设 1）                |
| #SBATCH -n 1               | 任务数                       |
| #SBATCH --cpus-per-task=4  | 每任务 CPU 核数                |
| #SBATCH --mem=16g          | 总内存                       |
| #SBATCH --gres=gpu:type:N  | GPU 请求（类型和数量）             |
| #SBATCH --constraint="..." | 硬件约束（如 a100-80G）          |
| #SBATCH --array=0-99%10    | Array Job, % 后为最大并发数      |
| #SBATCH --output=%x.%j.out | 标准输出文件                    |
| #SBATCH --error=%x.%j.err  | 标准错误文件                    |
| #SBATCH --mail-type=FAIL   | 邮件通知类型                    |
| #SBATCH --mail-user=email  | 通知邮箱                      |

### 不要在集群上存放受限数据

HIPAA（医疗数据）、FERPA（学生记录）等受保护的信息**禁止**存放在 HPC 集群上。如果你不确定你的数据是否属于受限类别，先联系 [tts-research@tufts.edu](mailto:tts-research@tufts.edu) 确认。

### 不要在登录节点上跑计算

在登录节点上直接跑 `python train.py` 会影响所有其他用户，管理员可能会直接终止你的进程。

### 不要请求远超实际需要的资源

“反正是免费的，我多请求一点”——这个想法是错的。你请求的资源在作业运行期间是**独占的**，别人无法使用。请求 8 块 GPU 但只用了 1 块，意味着 7 块 GPU 白白浪费。用 `seff` 检查实际用量，下次按需请求。

### 注意软件许可证

集群上有些软件**不是开源的**（如 MATLAB、Mathematica），使用时需要遵守对应的许可协议。违反许可协议算违反使用政策。

### 不要用 HPC 做商业用途

集群只能用于 Tufts 相关的学术和研究活动。不能用来给外部公司跑计算、挖矿、或者任何商业盈利活动。

### 毕业或离职前备份你的数据

你离开 Tufts 后，账号访问权限会被取消。**提前**把需要的数据下载到本地或其他存

储，不要等到最后一天。

### 你在集群上的活动不是完全私密的

Tufts 不会日常监控你的使用，但系统会记录日志。如果有合理理由怀疑违规，管理员可以**不通知你的情况下**查看你的活动记录 and 文件。

#### 提示

以上规则的核心精神其实就一条：**这是一个共享资源，你的行为会影响其他人。**按需使用、不浪费、不越界，就不会有问题。